App Development-2 Project Report

App Dev-2 Project Report

1. Student Details

Name: Akshayan Vishaal B
Roll Number: 24f2007248
Email: 24f2007248@ds.study.iitm.ac.in
About Me: I am an aspiring Computer Science engineer pursuing the IIT Madras BS Degree Program. This project provided me with an opportunity to apply the concepts learned in my coursework to a real world application. Through this experience, I strengthened my practical understanding of web technologies and backend systems.

---

2. Project Details

Project Title: Trekking Management App V2

Problem Statement:

This project involves developing a Trekking Management Application using SQLite for database storage and HTML/CSS for the user interface. The application manages treks, bookings, staff, and trekkers by storing and retrieving information from the database. Dynamic frontend pages allow users and administrators to perform operations such as creating, updating, deleting, and viewing records. The system also includes asynchronous and scheduled background tasks, such as CSV export and automated email notifications, along with Redis-based caching to improve application performance.

Approach:

Develop a role-based web application using Flask (API), Vue.js (UI), SQLite (database), and Redis (caching). Implement separate modules for Admin, Trek Staff, and Users, enabling trek management, staff assignment, bookings, slot tracking, search/filtering, and reporting through secure REST APIs and a responsive interface.

---

3. AI/LLM Declaration

I used **ChatGPT** to assist in writing UI Layouts and Bootstrap enhancements. The extent of AI/LLM usage is around **25%**, limited to **code suggestions and documentation formatting**. All final implementation logic, debugging, and integration were done manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Bootstrap 5	Frontend styling and responsive design
Vue.js	Frontend user interface
Redis	An in-memory data store used as the Celery message broker and result backend, and also used for caching
Celery	A distributed task queue used to execute long-running and scheduled background tasks
SQLite	Lightweight local database for storing user data
SMTP	Python compatible protocol used to send emails for mailing based tasks
Werkzeug	Password hashing and security utilities
Python datetime	Handling dates and timestamps for records

---

5. Database Schema / ER Diagram

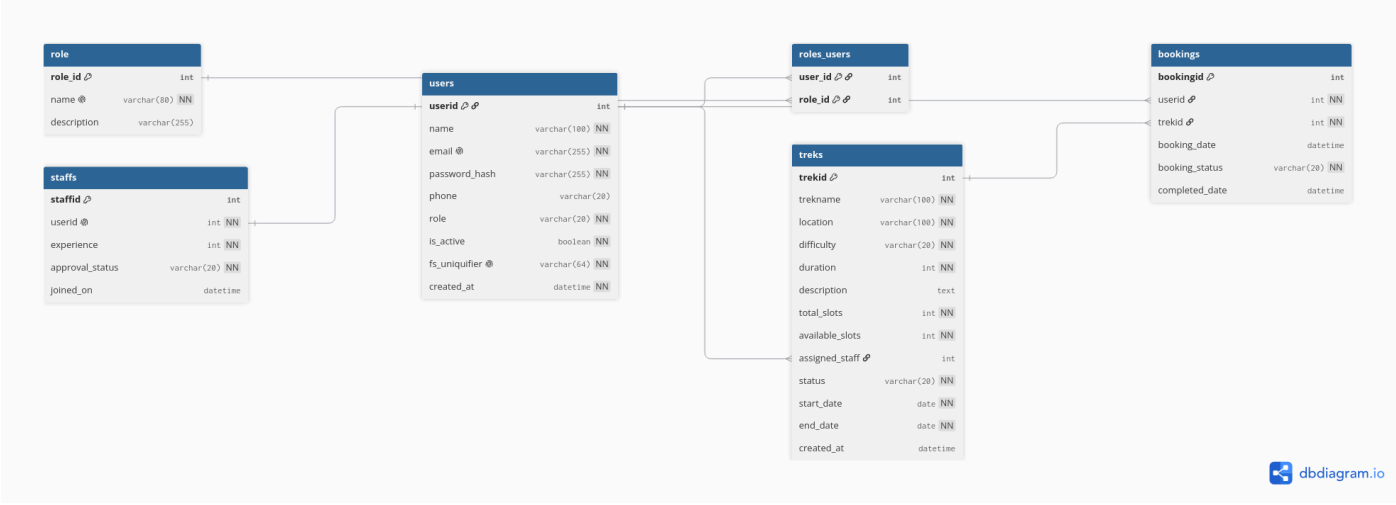
Tables:

1. **User**: Stores user account details including personal information and login credentials.

    - `(userid, name, email, password_hash, phone, role, is_active, fs_uniquifier, created_at)`
2. **Role**: Stores the available user roles used for role-based access control.
    - `(role_id, name, description)`
3. **RoleUser**: Association table that links users with roles in a many-to-many relationship.
    - (user_id, role_id)
4. **StaffProfile**: Stores additional information for users registered as trekking staff.
    - `(staffid, userid, experience, approval_status, joined_on)`
5. **Trek**: Stores details of trekking events including location, schedule, capacity, and assigned staff.
    - `(trekid, trekname, location, difficulty, duration, description, total_slots, available_slots, assigned_staff, status, start_date, end_date, created_at)`
6. **Booking**: Stores booking records linking users with trekking events.
    - (bookingid, userid, trekid, booking_date, booking_status, completed_date)

Relationships:

- Many-to-Many → `User ↔ Role`
- One-to-Many → `User → StaffProfile`
- One-to-Many → `User → Booking`
- One-to-Many → `StaffProfile → Trek`
- One-to-Many → `Trek → Booking`



6. API Resource Endpoints

Endpoint	Method	Description
`/login`	GET, POST	Authenticate user and create a session.
`/logout`	GET	Logout the current user and clear the session.
`/register`	GET, POST	Register a new trek user account.
`/staff/register`	GET, POST	Register a new trek staff account awaiting admin approval.
`/admin/dashboard`	GET	Display administrator dashboard statistics and recent bookings.
`/admin/treks`	GET	View and search all treks with filters and pagination.
`/admin/add_trek`	GET, POST	Create a new trek.
`/admin/edit_trek/<trek_id>`	GET, POST	Edit an existing trek.
`/admin/delete_trek/<trek_id>`	POST	Delete a trek.
`/admin/staff`	GET	View pending, approved and blacklisted staff members.
`/admin/update_staff_status/<staff_id>/<action>`	POST	Approve, reject, blacklist or restore a staff member.
`/admin/users`	GET	View and search registered users.
`/admin/toggle_user_status/<user_id>`	POST	Activate or deactivate a user account.
`/admin/bookings`	GET	View and search all trek bookings.
`/staff/dashboard`	GET	Display staff dashboard with assigned trek statistics.

`/staff/treks`	GET	View assigned treks with search and status filter.
`/staff/manage_trek/<trek_id>`	GET	View trek details and participants.
`/staff/update_trek_config/<trek_id>`	POST	Update available slots and trek status.
`/staff/change_status/<trek_id>/<next_state>`	POST	Mark a trek as Started or Completed.
`/staff/participants`	GET	View participants with search and filters.
`/staff/profile`	GET, POST	View and update staff profile information.
`/user/dashboard`	GET	Display user dashboard with available treks and recent bookings.
`/user/treks`	GET	Browse available treks with search, difficulty filter and pagination.
`/user/book_trek/<trek_id>`	GET, POST	View trek details and book a trek.
`/user/cancel_booking/<trek_id>`	POST	Cancel an existing booking and restore the available slot.
`/user/bookings`	GET	View user bookings and trek history with search and filters.
`/user/profile`	GET, POST	View and update user profile information.

---

7. Architecture and Features

Architecture Overview:

- **Login and Registration:** Flask-based authentication verifies user credentials stored in the database. The admin account is pre-seeded, staff accounts are created by the admin, and users can self-register and log in.
- **Admin Dashboard:** Admins can manage treks, staff, and users, assign staff to treks, search records, blacklist accounts, and view all bookings.
- **Staff Dashboard:** Staff can manage only their assigned treks, update trek details, and manage registered participants.
- **User Dashboard:** Users can browse, search, and book available treks, view trekking history, and update their profiles.
- **CSV Export:** Users can export trekking history as a CSV file through an asynchronous Celery–Redis task.
- **Reminders and Reports:** Daily reminder emails are sent for upcoming treks, and monthly HTML reports with trekking statistics are emailed to the admin via SMTP.
- **Caching:** Redis caching is used for dashboards and trek listing pages, with a cache expiry of 5 minutes.

Implemented Features:

- User, Staff, and Admin authentication with secure password hashing.
- Role-based dashboards for Admin, Staff, and Trek Users.
- Trek creation, modification, deletion, and management by the administrator.
- Trek staff creation, assignment, management, and blacklist/deactivation.
- User registration, profile management, and session-based authentication.
- Trek browsing with search, filtering, and pagination.
- Trek booking and booking cancellation with automatic slot management.
- Duplicate booking prevention and overbooking prevention.
- Staff management of assigned treks and participant information.
- Trek status tracking (Open, Closed, Started, and Completed).
- Booking history and status tracking (Booked, Cancelled, and Completed) for users.
- Search, filtering, and pagination across admin, staff, and user management pages.
- Asynchronous CSV export of trekking history using Celery and Redis.
- Daily reminder emails and monthly HTML activity reports via SMTP.
- Redis-based caching with cache expiry and invalidation for trek listing APIs.
- SQLite database integration using SQLAlchemy ORM for data persistence.

---

8. Video Presentation

Drive Link: [24F2007248 Video Presentation]

https://drive.google.com/file/d/14IO3EPtq5Shh5K72ZRtqPt8pjMSQVtrZ/view?usp=sharing

(Accessible to all with “View” permission.)
